

Énoncé : La Plus Longue Sous-Suite Commune (LCS)

Contexte

Dans le domaine de la comparaison de textes, de la bio-informatique (analyse d'ADN) ou de la gestion de versions (comme la commande `diff` de Git), on cherche souvent à mesurer la similarité entre deux chaînes de caractères. Une méthode classique consiste à trouver leur **Plus Longue Sous-Suite Commune** (LCS pour *Longest Common Subsequence*).

Définition d'une sous-suite

Une sous-suite est une suite de caractères qui apparaît dans le même ordre dans les deux chaînes, mais **pas nécessairement de manière consécutive** (contrairement à un sous-mot/facteur).

- *Exemple* : Dans "CHAMPIGNON" et "CAMPING", la séquence "CAMPIN" est une sous-suite commune de longueur 6. Les lettres s'y retrouvent dans le bon ordre, même si elles sont séparées par d'autres caractères.

Objectif

Écrire une fonction `lcs(chaine1, chaine2)` en Python qui détermine la **longueur maximale** d'une sous-suite commune aux deux chaînes fournies.

Consignes de codage

- **chaine1** : Une chaîne de caractères (string).
- **chaine2** : Une autre chaîne de caractères (string).
- **Valeur de retour** : Un **entier** représentant la longueur de la LCS.

⚠ Contraintes algorithmiques majeures :

- **Respect de l'ordre** : L'ordre des caractères doit être strictement préservé. Par exemple, la LCS entre "AB" et "BA" est de longueur 1 (soit "A", soit "B"), car on ne peut pas inverser les lettres.
- **Le piège des choix destructeurs** : Votre algorithme ne doit pas être "court-termiste". Parfois, sacrifier une lettre unique au début permet de valider un énorme bloc de caractères identiques plus loin (voir l'exemple de l'Apocalypse ci-dessous).
- **Performance (Programmation Dynamique requise)** : Une approche récursive naïve possède une complexité exponentielle de $O(2^{n+m})$. Votre fonction doit être optimisée (par exemple avec un tableau 2D de programmation dynamique) pour être capable de traiter des chaînes de plusieurs centaines de caractères en une fraction de seconde.

Exemples de comportement attendu

Exemple 1 : Lettres non consécutives

```
print(lcs("LOGIQUE", "ALGORITHME"))  
# Sortie attendue : 4 (La sous-suite est "L-G-O-R" ou autre combinaison de
```

```
longueur 4)
```

Exemple 2 : Le Test de l'Apocalypse (Choix des blocs vs Lettres isolées)

Imaginons deux chaînes construites ainsi :

- Chaîne 1 : "A" suivi de 40 "B", puis "C", puis 40 "D", etc.
- Chaîne 2 : 40 "B" suivis de "A", puis 40 "D" suivis de "C", etc.

Si l'algorithme s'obstine à vouloir aligner les lettres uniques au début ("A", "C"...), il passera à côté des immenses blocs de "B" et de "D". L'optimum global consiste ici à ignorer les lettres isolées pour cumuler les grands blocs.

Prototype de la fonction

```
def lcs(chaine1: str, chaine2: str) -> int:
    """
    Calcule la longueur de la plus longue sous-suite commune entre deux chaînes.

    :param chaine1: Première chaîne de caractères (str)
    :param chaine2: Seconde chaîne de caractères (str)
    :return: Longueur de la LCS (int)
    """
    # Votre code ici
    pass
```

Tests

```
print("Démarrage des tests pour la Plus Longue Sous-Suite Commune (LCS)...")

# Test 1 : Cas de base - Mots identiques
print("Test 1...", lcs("PYTHON", "PYTHON"))
assert lcs("PYTHON", "PYTHON") == 6, "Échec Test 1 : Deux mots identiques doivent renvoyer leur longueur totale"

# Test 2 : Aucun point commun
print("Test 2...", lcs("ABC", "XYZ"))
assert lcs("ABC", "XYZ") == 0, "Échec Test 2 : Aucun caractère commun doit renvoyer 0"

# Test 3 : Un mot est vide
print("Test 3...", lcs("", "TEST"))
assert lcs("", "TEST") == 0, "Échec Test 3 : Si un des mots est vide, le résultat doit être 0"
```

```
# Test 4 : Inclusion totale (Un mot est caché dans l'autre)
print("Test 4...", lcs("AC", "ABC"))
assert lcs("AC", "ABC") == 2, "Échec Test 4 : 'AC' est entièrement présent dans 'ABC'"

# Test 5 : Lettres consécutives vs non-consécutives (L-G-0)
print("Test 5...", lcs("LOGIQUE", "ALGORITHME"))
assert lcs("LOGIQUE", "ALGORITHME") == 4, "Échec Test 5 : Problème avec les lettres espacées (L-G-0)"

# Test 6 : L'exemple du cours (C-A-M-P-I-N-G)
print("Test 6...", lcs("CHAMPIGNON", "CAMPING"))
assert lcs("CHAMPIGNON", "CAMPING") == 6, "Échec Test 6 : L'exemple classique 'CAMPING' a échoué"

# Test 7 : Piège des inversions d'ordre
print("Test 7...", lcs("AB", "BA"))
assert lcs("AB", "BA") == 1, "Échec Test 7 : L'algorithme doit respecter l'ordre des lettres (soit 'A', soit 'B')"

# Test 8 : Performance (Chaînes plus longues - Risque d'explosion 2^n sans DP)
grand_mot1 = "ANANAS_" * 50 # 350 caractères
grand_mot2 = "BANANE_" * 50 # 350 caractères
print("Test 8 (Test de performance)...", lcs(grand_mot1, grand_mot2))
assert lcs(grand_mot1, grand_mot2) == 250, "Échec Test 8 : L'algorithme est trop lent ou gère mal les grands volumes"

# Test 9 : Le Test de l'Apocalypse (Chaos total et choix destructeurs)
mot_piege_1 = "A" + "B"*40 + "C" + "D"*40 + "E" + "F"*40 + "G"
mot_piege_2 = "B"*40 + "A" + "D"*40 + "C" + "F"*40 + "E" + "G"

print("Test 9 (Le Test de l'Apocalypse)...", lcs(mot_piege_1, mot_piege_2))
# Explication :
# Soit on apparie les lettres uniques (A, C, E, G) -> Longueur 4
# Soit on prend les blocs de lettres répétées (B*40 + D*40 + F*40 + G) -> Longueur 121
assert lcs(mot_piege_1, mot_piege_2) == 121, "Échec Test 10 : L'algorithme a sacrifié les grands blocs pour sauver les premières lettres uniques !"

print("\nTous les tests LCS sont passés avec succès !")
```